

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Операционные системы и системное программирование

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

Симулятор мини-ЭВМ PDP-11
БГУИР КП 6-05-0611-05 111 ПЗ

Студент: группы 450501,
Лебедь М. В.

Руководитель: старший
преподава-тель каф. ЭВМ
Поденок Л. П.

Минск 2026

Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Факультет компьютерных систем и сетей

УТВЕРЖДАЮ
Заведующий кафедрой
_____ Б.В.Никульшин
« ____ » _____ 2026 г.

ЗАДАНИЕ
по курсовому проектированию

Студенту Лебедю Михаилу Владимировичу

1. Тема проекта: симулятор мини-ЭВМ PDP-11.
2. Срок сдачи студентом законченного проекта: 15 мая 2026 г.
3. Исходные данные к проекту: язык программирования С
4. Содержание расчетно-пояснительной записки
Введение. 1. Обзор литературы. 2. Постановка задачи. 3. Системное проектирование. 4. Функциональное проектирование. 5. Результат работы. Заключение. Список использованных источников. Приложения
5. Перечень графического материала (с точным обозначением обязательных чертежей и графиков):
 1. Схема взаимодействия пользователя с симулятором мини-ЭВМ PDP-11
 2. Схема алгоритма fetch operand(mode, reg).
 3. Ведомость документов
6. Консультант по проекту Поденок Л. П.
7. Календарный график работы над проектом на весь период проектирования (с обозначением сроков выполнения и трудоемкости отдельных этапов):
раздел 1, 2 к 01.03 – 15 %;
разделы 3, 4 к 01.04 – 50 %;
разделы 5, 6, 7, 8, 9 к 01.05 – 80 %;
оформление пояснительной записки и графического материала к 15.05 – 100%
Защита курсового проекта с 29.05 по 09.06.

РУКОВОДИТЕЛЬ

Л.П. Поденок

Задание принял к исполнению

М.В. Лебедь

СОДЕРЖАНИЕ

Введение	4
1 Обзор литературы	5
1.1 Обзор аналогов	5
1.2 Постановка задачи	5
2 Системное проектирование	7
2.1 Модуль ядра процессора (<i>cpu</i>)	7
2.2 Модуль памяти (<i>memory</i>)	7
2.3 Модуль декодирования адресации (<i>decode</i>)	7
2.4 Модуль системы команд (<i>instructions</i>)	8
2.5 Модуль загрузки программ (<i>loader</i>)	8
2.6 Модуль дизассемблера (<i>disasm</i>)	8
2.7 Модуль отладки (<i>debug</i>)	8
3 Функциональное проектирование	9
3.1 Структура состояния процессора	9
3.2 Режимы адресации	9
3.3 Система команд	10
3.4 Форматы загрузки программ	10
4 Разработка программных модулей	11
4.1 Разработка схем алгоритмов	11
4.2 Разработка алгоритмов	11
5 Результат работы	13
5.1 Сборка проекта	13
5.2 Тестирование программы	13
Заключение	15
Список использованных источников	16
Приложение А	17
Приложение Б	18
Приложение В	19

ВВЕДЕНИЕ

Архитектура ЭВМ PDP-11 компании Digital Equipment Corporation оказала значительное влияние на развитие вычислительной техники и операционных систем. Именно на базе PDP-11 были разработаны первые версии операционной системы UNIX, а архитектурные решения данной машины нашли отражение в последующих поколениях процессоров. Изучение архитектуры и системы команд PDP-11 является важной задачей в контексте образования в области системного программирования.

Одним из эффективных методов изучения архитектуры является разработка программного симулятора, реализующего поведение реального процессора. Такой симулятор позволяет исследовать систему команд, режимы адресации и принципы работы процессора без использования физического оборудования. Программная реализация также предоставляет возможности для трассировки выполнения программ, просмотра состояния регистров и памяти.

В рамках данного курсового проекта разработан симулятор архитектуры PDP-11 на языке программирования C. Симулятор реализует ядро процессора с поддержкой всех восьми режимов адресации, набором из 46 команд, дизассемблером и средствами отладки. Разрабатываемая система представляет собой консольное приложение для POSIX-совместимых операционных систем.

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Обзор аналогов

Для оценки разрабатываемого симулятора необходимо рассмотреть существующие программные реализации архитектуры PDP-11. В настоящее время существует ряд широко известных эмуляторов и симуляторов данной архитектуры, каждый из которых обладает определёнными преимуществами и ограничениями.

Наиболее распространённым является симулятор SIMH, разработанный Бобом Супником. Данное решение обеспечивает высокую точность эмуляции и поддерживает множество периферийных устройств, включая дисковые накопители, терминалы и сетевые адаптеры. Однако сложность архитектуры SIMH затрудняет его использование в образовательных целях.

Другим известным решением является эмулятор PDP11GUI, предоставляющий графический интерфейс для работы с виртуальной машиной PDP-11. Несмотря на удобство визуального представления, данная программа требует значительных системных ресурсов и ориентирована на пользователей с опытом работы с архитектурой.

Также существуют учебные реализации на различных языках программирования, публикуемые на образовательных платформах. Как правило, такие проекты реализуют ограниченный набор команд и не содержат средств отладки.

Таким образом, существующие решения либо избыточно сложны для образовательного использования, либо не предоставляют полноценных инструментов исследования архитектуры. Это подтверждает актуальность разработки компактного симулятора с полным набором команд, средствами трассировки и дизассемблером.

1.2 Постановка задачи

После рассмотрения существующих аналогов можно сделать вывод о необходимости разработки симулятора PDP-11, ориентированного на изучение архитектуры и системного программирования. В рамках данного курсового проекта поставлены следующие задачи:

1) Реализовать ядро процессора PDP-11 с поддержкой регистров общего назначения R_0-R_5 , указателя стека SP и счётчика команд PC, а также слова состояния процессора PSW.

2) Разработать модуль декодирования операндов с поддержкой всех восьми режимов адресации PDP-11: регистрового, косвенного, автоинкремента, автодекремента, индексного и их косвенных вариантов.

3) Реализовать набор из 46 команд, включая двухоперандные (MOV, ADD, SUB, CMP, BIT, BIC, BIS), однооперандные (CLR, COM, INC, DEC,

NEG, ADC, SBC, TST, ROL, ROR, ASL, ASR, SWAB), команды ветвления и управления флагами.

4) Разработать загрузчик программ с поддержкой трёх форматов: массива слов, двоичного файла с заголовком и формата SIMH paper tape.

5) Реализовать дизассемблер, обеспечивающий преобразование машинных кодов PDP-11 в символьное представление команд с указанием режимов адресации.

6) Разработать средства отладки: дампы регистров, дампы памяти и трассировку выполнения программ.

7) Провести тестирование всех реализованных компонентов с использованием автоматических тестов.

В качестве языка программирования выбран C (стандарт C11), поскольку он обеспечивает прямой доступ к системным ресурсам и широко используется в системном программировании. Разрабатываемый симулятор представляет собой консольное приложение для POSIX-совместимых операционных систем.

2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

После определения требований к симулятору необходимо выполнить разбиение системы на функциональные модули. Модульная структура упрощает разработку, тестирование и возможное расширение программы. Каждый модуль реализован в виде пары файлов: заголовочного (.h) и файла реализации(.c).

2.1 Модуль ядра процессора (cpu)

Модуль ядра процессора (cpu.c, cpu.h) реализует центральное исполнительное устройство симулятора. В нём определены функции инициализации и сброса процессора, пошагового выполнения инструкций, а также операций над флагами слова состояния процессора PSW.

Функция `cpu_step()` реализует один такт работы процессора: читает слово из памяти по адресу PC, инкрементирует PC на 2, передаёт слово диспетчеру инструкций и увеличивает счётчик циклов. Функция `cpu_run()` запускает цикл выполнения с трассировкой и контролем лимита тактов.

Вспомогательные функции `cpu_set_flag()`, `cpu_clear_flag()` и `cpu_get_flag()` обеспечивают операции с отдельными битами PSW. Функция `cpu_update_nz()` обновляет флаги N и Z по результату операции.

2.2 Модуль памяти (memory)

Модуль памяти (memory.c, memory.h) предоставляет интерфейс для чтения и записи байтов и слов в адресное пространство симулятора объёмом 65536 байт. Реализованы функции `mem_read_byte()`, `mem_read_word()`, `mem_write_byte()` и `mem_write_word()`.

Чтение и запись слов выполняются в формате `little-endian`, соответствующем архитектуре PDP-11: младший байт хранится по меньшему адресу. Физическая память представлена массивом `uint8_t` размером `MEM_SIZE = 65536` байт, являющимся полем структуры PDP11.

2.3 Модуль декодирования адресации (decode)

Модуль декодирования (decode.c, decode.h) реализует все восемь режимов адресации архитектуры PDP-11. Ключевыми функциями модуля являются `fetch_operand()`, `resolve_dst_addr()` и `store_operand()`, обеспечивающие чтение исходного операнда, определение адреса назначения и запись результата соответственно.

Макросы `INSTR_SRC_MODE`, `INSTR_SRC_REG`, `INSTR_DST_MODE` и `INSTR_DST_REG` извлекают соответствующие поля из 16-битного кода

инструкции. Макрос `INSTR_BRANCH_OFFSET` извлекает смещение перехода со знаком.

2.4 Модуль системы команд (`instructions`)

Модуль системы команд (`instructions.c, instructions.h`) реализует 46 инструкций архитектуры PDP-11, сгруппированных по типам: двухоперандные, однооперандные, команды ветвления, управления флагами, вызова подпрограмм и прерываний. Точкой входа является функция `execute()`, принимающая указатель на процессор и 16-битный код инструкции. Диспетчер инструкций последовательно анализирует различные группы битов кода команды: биты 15–12 для двухоперандных команд, биты 15–6 для однооперандных, биты 15–8 для команд ветвления.

2.5 Модуль загрузки программ (`loader`)

Модуль загрузки (`loader.c, loader.h`) реализует три формата загрузки программ в память симулятора. Функция `loader_load_words()` загружает программу из массива 16-битных слов, задаваемого непосредственно в тестовом коде. Функции `loader_load_raw()` и `loader_load_header()` загружают двоичные файлы без заголовка и с шестибайтным заголовком соответственно. Функция `loader_load_tape()` реализует загрузку в формате SIMH paper tape.

2.6 Модуль дизассемблера (`disasm`)

Модуль дизассемблера (`disasm.c, disasm.h`) обеспечивает преобразование машинного кода PDP-11 в символьное представление. Функция `disassemble()` принимает адрес инструкции в памяти симулятора и записывает её текстовое представление в предоставленный буфер.

Вспомогательная функция `decode_operand()` преобразует пару (`mode, reg`) в текстовую форму операнда с учётом специальных обозначений для SP и PC, а также указывает дополнительные слова при индексном режиме.

2.7 Модуль отладки (`debug`)

Модуль отладки (`debug.c, debug.h`) предоставляет инструменты для исследования состояния симулятора. Функция `debug_dump_regs()` выводит содержимое всех восьми регистров и слова состояния PSW с расшифровкой флагов N, Z, V, C. Функция `debug_dump_mem()` выводит дампы заданного диапазона памяти в восьмеричном формате. Функция `debug_trace()` выводит строку трассировки для текущей инструкции.

3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

В данном разделе рассматриваются основные структуры данных и компоненты программы, определяющие способ представления состояния процессора, организацию системы команд и механизмы взаимодействия модулей.

3.1 Структура состояния процессора

Центральной структурой данных программы является *PDP11*, определённая в файле `types.h`:

```
typedef struct {
    uint16_t reg[8];
    uint16_t psw;
    uint8_t  memory[MEM_SIZE];
    int      running;
    uint64_t cycles;
} PDP11;
```

Поле `reg` хранит восемь 16-битных регистров процессора: R_0 – R_5 (регистры общего назначения), R_6 (указатель стека *SP*) и R_7 (счётчик команд *PC*). Поле `psw` содержит слово состояния процессора с флагами *C* (перенос), *V* (переполнение), *Z* (ноль) и *N* (знак). Поле `memory` представляет адресное пространство объёмом 65536 байт. Поля `running` и `cycles` используются для управления циклом выполнения и подсчёта числа исполненных инструкций.

3.2 Режимы адресации

Архитектура *PDP-11* поддерживает восемь режимов адресации, кодируемых тремя битами поля `mode` в коде инструкции. В таблице 3.1 приведено описание реализованных режимов.

Таблица 3.1 – Режимы адресации *PDP-11*

Режим	Название	Синтаксис	Описание
0	Регистровый	R_n	Значение в регистре
1	Косвенный	(R_n)	Значение по адресу из регистра
2	Автоинкремент	$(R_n)+$	Адрес из регистра, затем $R_n += 2$
3	Косв. автоинкремент	$@(R_n)+$	Двойная косвенность с инкрементом
4	Автодекремент	$-(R_n)$	$R_n -= 2$, затем адрес из регистра
5	Косв. автодекремент	$@-(R_n)$	Двойная косвенность с декрементом
6	Индексный	$X(R_n)$	Адрес = R_n + следующее слово
7	Косвенный индексный	$@X(R_n)$	Косвенность через индексный адрес

При использовании РС (R₇) в качестве регистра базы режимы 2 и 3 реализуют непосредственную адресацию (#N и @#N), а режимы 6 и 7 – относительную (E и @E).

3.3 Система команд

Реализованные команды разбиты на следующие группы: двухоперандные команды (MOV, CMP, BIT, BIC, BIS, ADD, SUB), однооперандные команды (CLR, COM, INC, DEC, NEG, ADC, SBC, TST, ROL, ROR, ASL, ASR, SWAB), команды ветвления (BR, BNE, BEQ, BPL, BMI, BVC, BVS, BCC, BCS, BGE, BLT, BGT, BLE, BHI, BLOS), команды управления флагами (CLC, CLV, CLZ, CLN, SEC, SEV, SEZ, SEN), команды вызова подпрограмм (JSR, RTS), команды цикла (SOB), команды прерываний (TRAP, EMT, RTI) и команда остановки (HALT).

3.4 Форматы загрузки программ

Загрузчик поддерживает три формата. Формат «массив слов» предназначен для прямой загрузки из тестового кода: принимает указатель на массив `uint16_t` и число слов. Формат «с заголовком» использует шестибайтный заголовок: адрес загрузки (2 байта), стартовый адрес (2 байта), размер данных (2 байта). Формат `SIMH paper tape` представляет собой блочную структуру с контрольными суммами, совместимую со стандартными образами ПО для PDP-11.

4 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

В данном разделе рассмотрены алгоритмы работы ключевых функций симулятора: диспетчера инструкций `execute()` и функции пошагового выполнения `cpu_step()`.

4.1 Разработка схем алгоритмов

Функция `execute()` реализует диспетчеризацию по коду инструкции. Схема алгоритма функции представлена в приложении А.

Функция `fetch_operand()` выполняет чтение значения операнда с учётом режима адресации. При режиме 0 значение возвращается непосредственно из регистра. Для остальных режимов вычисляется адрес и производится чтение из памяти.

4.2 Разработка алгоритмов

Алгоритм по шагам функции `execute()`:

Шаг 1) Начало;

Шаг 2) Входные данные: `cpu` – указатель на состояние процессора; `instr` – 16-битный код инструкции;

Шаг 3) Проверка: если `instr == 0000000`, вызвать `instr_halt()` и завершить;

Шаг 4) Проверка: если `instr == 0000002`, вызвать `instr_rti()` и завершить;

Шаг 5) Проверка: если `instr` в диапазоне 0000240–0000277, вызвать `instr_flags()` и завершить;

Шаг 6) Проверка: если биты 15–9 равны 0077, вызвать `instr_sob()` и завершить;

Шаг 7) Извлечь биты 15–12 (`opcode`). По значению `opcode` выбрать двухоперандную команду (`MOV`, `CMP`, `BIT`, `BIC`, `BIS`, `ADD`, `SUB`);

Шаг 8) Извлечь биты 15–6 (`opcode`). По значению `opcode` выбрать однооперандную команду;

Шаг 9) Проверка маски для `JSR` (`0xFE00 == 0x0800`), при совпадении вызвать `instr_jsr()`;

Шаг 10) Проверка маски для `RTS` (`0xFFF8 == 000200`), при совпадении вызвать `instr_rts()`;

Шаг 11) Извлечь биты 15–8 (`opcode`). По значению выбрать команду ветвления или прерывания;

Шаг 12) Если команда не распознана – вывести сообщение об ошибке, установить `running = 0`;

Шаг 13) Конец.

Алгоритм по шагам функции `cpu_step()`:

Шаг 1) Начало;

Шаг 2) Входные данные: `cpu` – указатель на состояние процессора;

Шаг 3) Прочитать слово из памяти по адресу `cpu->reg[PC]`: `instr = mem_read_word(cpu, cpu->reg[PC]);`
 Шаг 4) Инкрементировать счётчик команд: `cpu->reg[PC] += 2;`
 Шаг 5) Передать код инструкции диспетчеру: `execute(cpu, instr);`
 Шаг 6) Увеличить счётчик тактов: `cpu->cycles++;`
 Шаг 7) Конец.

Алгоритм по шагам вспомогательной функции `resolve_addr()`:

Шаг 1) Начало;
 Шаг 2) Входные данные: `cpu` – состояние процессора; `mode` – режим адресации (0–7); `reg` – номер регистра (0–7);
 Шаг 3) В зависимости от `mode`:
 Шаг 4) `mode == 1`: `addr = cpu->reg[reg];`
 Шаг 5) `mode == 2`: `addr = cpu->reg[reg]; cpu->reg[reg] += 2` (автоинкремент);
 Шаг 6) `mode == 3`: `addr = cpu->reg[reg]; cpu->reg[reg] += 2; addr = mem_read_word(cpu, addr)` (косвенный автоинкремент);
 Шаг 7) `mode == 4`: `cpu->reg[reg] -= 2; addr = cpu->reg[reg]` (автодекремент);
 Шаг 8) `mode == 5`: `cpu->reg[reg] -= 2; addr = mem_read_word(cpu, cpu->reg[reg])` (косвенный автодекремент);
 Шаг 9) `mode == 6`: `index = mem_read_word(cpu, PC); PC += 2; addr = cpu->reg[reg] + index` (индексный);
 Шаг 10) `mode == 7`: аналогично `mode 6`, затем `addr = mem_read_word(cpu, addr)` (косвенный индексный);
 Шаг 11) Вернуть `addr`;
 Шаг 12) Конец.

5 РЕЗУЛЬТАТ РАБОТЫ

5.1 Сборка проекта

Для компиляции симулятора используется `Makefile` с отдельной компиляцией модулей. Исходные файлы расположены в каталоге `src/`, объектные файлы и исполняемый файл создаются в каталоге `build/`. Сборка выполняется командой:

```
$ make
```

`Makefile` использует компилятор `gcc` с флагами `-Wall -Wextra -std=c11 -g`.

При необходимости полной пересборки применяется:

```
$ make rebuild
```

Для запуска скомпилированной программы:

```
$ make run  
$ ./build/pdp11
```

5.2 Тестирование программы

В ходе разработки симулятора проведено тестирование всех реализованных компонентов с использованием автоматических тестов, организованных в файле `main.c`. Каждый тест формирует программу в виде массива слов, загружает её в память симулятора, выполняет и проверяет ожидаемое состояние регистров.

Тест команды `ADD` проверяет сложение двух регистровых значений и неизменность источника:

```
=====
Быстрый тест: asm_programs/test2_add.asm
=====
=== ПРОХОД 1 (asm_programs/test2_add.asm) ===
```

```
--- ИТОГИ ПРОХОДА 1 ---
```

```
Найдено меток: 1
```

```
Размер кода: 528 байт (001020)
```

```
Последний адрес: 001020
```

```
-----
```

```
=== Проход 2 ===
```

Записано блоков: 1

Успех!

Выходной модуль : test2_add.lda

Листинг : test2_add.lst

Стартовый адрес : 001000

Размер кода : 528 байт

Таре: загрузка 'test2_add.lda'

Блок 0: 16 байт по адресу 001000

Стартовый адрес: 001000

Таре: загружено 1 блоков, 16 байт, старт 001000

HALT

=== Регистры ===

R0 (): 000110 (72)

R1 (): 000030 (24)

R2 (): 000000 (0)

R3 (): 000000 (0)

R4 (): 000000 (0)

R5 (): 000000 (0)

R6 (SP): 177776 (65534)

R7 (PC): 001020 (528)

PSW: 000000 [N=0 Z=0 V=0 C=0]

Cycles: 5

=== РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ===

RESULT_R0=000110

RESULT_R1=000030

RESULT_R2=000000

RESULT_R3=000000

RESULT_R4=000000

RESULT_R5=000000

RESULT_SP=177776

RESULT_PC=001020

RESULT_N=0

RESULT_Z=0

RESULT_V=0

RESULT_C=0

=== КОНЕЦ РЕЗУЛЬТАТА ===

=====

Дизассемблер проверялся путём разбора кодов тестовых программ. Трассировка выполнения выводит для каждой инструкции адрес, мнемонику и текущие значения регистров:

```

[001000] MOV #100,R0      R0=000000 R1=000000 ... SP=177776
[001004] MOV R0,R1        R0=000100 R1=000000 ...
$001006] HALT             R0=000100 R1=000100 ...
SP=177776

```

Итоговый отчёт о тестировании подтвердил прохождение всех разработанных тестов без ошибок.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы были углублены знания языка программирования С и получены практические навыки системного программирования в POSIX-совместимых операционных системах. Освоены принципы работы архитектуры PDP-11, включая систему команд, режимы адресации и механизмы работы с памятью.

В результате работы разработан полноценный программный симулятор ЭВМ PDP-11, включающий ядро процессора с поддержкой восьми режимов адресации и 46 команд, модуль памяти объёмом 65536 байт, загрузчик программ с поддержкой трёх форматов, дизассемблер и средства отладки.

Реализованные средства трассировки и дампа регистров обеспечивают возможность пошагового исследования выполнения программ на уровне машинных кодов. Автоматические тесты подтвердили корректность реализации всех групп команд и режимов адресации.

Программа имеет потенциал для дальнейшего развития: добавление байтовых операций, реализация эмуляции терминала, поддержка блочных устройств, разработка встроенного отладчика и ассемблера, а также реализация блока управления памятью MMU для поддержки виртуальной адресации.

Данный курсовой проект предоставил практический опыт в области эмуляции процессорных архитектур, подчеркнув важность корректной работы с памятью, точного соответствия спецификации архитектуры и значение автоматизированного тестирования при разработке системного программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *PDP-11 Architecture Handbook*. – *Digital Equipment Corporation*, 1983. – 416 с.
2. *PDP-11 Processor Handbook. PDP-11/40*. – *Digital Equipment Corporation*, 1972. – 340 с.
3. Керниган Б., Ритчи Д. Язык программирования C. – 2-е изд. – М.: Вильямс, 2016. – 288 с.
4. Таненбаум Э. Современные операционные системы. – 4-е изд. – СПб.: Питер, 2019. – 1120 с.
5. *Lions J. Lions' Commentary on UNIX 6th Edition with Source Code*. – *Peer-to-Peer Communications*, 1996. – 254 с.

ПРИЛОЖЕНИЕ А

Схема взаимодействия пользователя с симулятором мини-ЭВМ PDP - 11

ПРИЛОЖЕНИЕ Б

Схема алгоритма `fetch_operand(mode, reg)`

ПРИЛОЖЕНИЕ В

Ведомость документов